Smart Home Sensor With Reasoning

Critical Design Review

Team EcoNest

Sutton Elliott

Shiv Ranchhod

William Xu

Evan Johnston

Tyler Rottermond

Sriharsha Jandhyala

Department of Computer Science

Texas A&M University

03/25/2026

Table of Contents

1 Introduction 2
1.1 General Scope and Problem Background 3
1.2 Needs Statement 3
1.3 Goals and Objectives 3
1.4 Design Constraints 4
1.5 Validation and Testing procedures 5
2 Proposed design 7
2.1 Updates to the proposal design (if applicable) 7
2.2 System description 9
2.3 Complete module-wise specifications 14
Updated Hardware Overview 21
Interfaces and Communication 21
Timing Behavior 22
Data Flow 22
3 Project management 22
3.1 Updated implementation schedule 22
3.2 Updated validation and testing procedures 25
Smart Plug Energy Monitoring 25
Sound Sensor Detection 25
End-to-End System Testing 25
Data Ingestion and Storage 25
System Performance Testing 26
AI/ML Validation 26
User Interface and Notification Testing 26
Overall System Validation 27
3.3 Updated division of labor and responsibilities 27
4 Preliminary results 29
5 References 30

1 Introduction

1.1 General Scope and Problem Background

Modern homes increasingly rely on smart devices such as **motion sensors**, **smart plugs**, and **environmental monitors** to collect data about **security**, and **energy consumption**. However, most existing smart home platforms only provide raw sensor readings or basic alerts without interpreting the information or prioritizing what matters most. As a result, homeowners must manually analyze scattered data from multiple devices, making it difficult to understand the overall state of their home or respond effectively to potential issues.

This lack of contextual reasoning is significant because households face real risks and inefficiencies that require timely interpretation. Idle energy consumption alone accounts for approximately **5–10 percent** of residential electricity use, often going unnoticed due to the absence of intelligent monitoring. At the same time, a burglary occurs roughly every **25 to 30 seconds** in the United States, demonstrating the importance of coordinated security insights rather than isolated device notifications. Although the global smart home market is projected to grow at a compound annual growth rate of about **27.07 percent** over the next five years, current systems still fail to transform device data into actionable understanding.

The scope of this project is to design and implement a centralized smart home monitoring system capable of collecting, integrating, and analyzing data from multiple sources. The system will be deployed across three experimental units of increasing complexity. Unit 1 and Unit 2 will consist of smart plugs for energy monitoring, along with motion and sound sensors for environmental awareness. Unit 3 will extend this capability by incorporating power monitoring at the breaker panel level to capture whole-home energy usage. All units will integrate with a home automation platform to enable coordinated monitoring and control.

1.2 Needs Statement

Over 94 million US households use smart security tools (Safehome, 2025), yet according to the FBI (Federal Bureau of Investigation), break-ins still occur every 26 seconds (FBI.gov, 2019). Additionally, the latest FBI Uniform Crime Reporting Statistics says that homes without security systems are 3 times more likely to be burglarized, (Alarms.org, 2019). From an energy perspective, smart thermostats claim to reduce costs from 10-15% per year (Globalnewswire.com, 2021). However, inactive smart home devices waste about 19 billion dollars annually (nrdc.org, 2015). Therefore, a system that **consolidates smart devices** is needed to **help homeowners better understand and respond to security risks and inefficient energy use** across their smart devices.

1.3 Goals and Objectives

Under normal conditions, it is up to **homeowners** to operate their home in a way that **conserves energy** and **provides security** to its occupants. The problem is that even if they

want to conserve energy or be secure, people might not know how, or forget, or maybe are just too lazy to operate their home in a way that does conserve energy and ensures security. Through **reasoning**, homeowners can be given **thorough suggestions** to **conserve energy** and **enhance security**.

Develop a **centralized smart home monitoring system** that applies data driven reasoning to optimize energy consumption and enhance home security.

In order to use **reasoning**, we have the following objectives:

- The smart home system should be able to **collect** different types of **data** for the **machine learning** models to **train** and make **predictions** with.
    - By having many **different types** of **data**, the **models** will be better trained to accomplish our **goal**.
- The smart home system should **quickly** and **securely** transfer collected **data**.
    - As we are working with **continuous** and **sensitive data**, we need to be able to **effectively** transfer that to the models.

Once accomplished our **objectives** pertaining to **reasoning** are:

- The system should be able to **detect anomalies** in the collected data.
    - This can be used to **enhance** the **security** of the occupants. They can **detect** power **issues** in appliances and possible **break-ins** and respond properly to ensure our goal is met.
- The system should be able to **analyze trends** in the collected data.
    - This will be used to make **predictions** on how the occupants will operate the home, and **automatically respond** based on these trends.
- The system should be able to provide **recommendations** to the occupants.
    - Using various data sources, the system should be able to provide recommendations to the occupants pertaining to **energy conservation** and **security** of their home.

Other objectives:

- The system should allow occupants to **access** collected **data** and be **informed** of any changes.
    - While we want the home to be **somewhat automated to** run considering **energy consumption** and **security**, the occupants should know about these changes in case of **wrong predictions** allowing the **occupants** to **change** the responses.

1.4 Design Constraints

Technical Constraints:

The **security** portion of our **system** relies on **motion** and **sound sensors**, while the **energy-monitoring devices** and **smart plugs** will provide the **data** for each of these focuses

on our design. This **data** will be provided to do a **machine learning** based **analysis** of the **security** and **energy** consumption. These **sensors** must reliably **capture** activity **patterns** and **events** that take place within their respective **environments** that can be processed and **stored** in a structured **database**. **Machine learning** techniques are then required to **analyze** this **data**, identify **abnormalities**, and **generate** meaningful **alerts** and **recommendations**. This all will be managed and **viewable** through a **mobile application**. The technical **design** must **ensure** that the sensor **data** is **accurate** and **consistent** to support the **AI system**, so that the **reasoning** behind it will be **accurate** and **reliable**.

Physical Constraints:

Motion and **sound** sensors will need to be physically **deployable** within **residential areas** and positioned to effectively **capture** room **activity** and not be intrusive after installation. Proper sensor **placement** is **essential** to ensure that the **data** collected **reflects** the actual data based upon the **energy consumption** and **security-related** events. **Poor placement** of these sensors will **negatively** impact the **machine learning** model and in turn the learning **accuracy**. Energy-monitoring devices also need to be placed at appropriate **power outlets** to provide **reliable** usage and energy consumption **data**. All **hardware** must be **compatible** with the overall **system** and non-invasive of the homeowners space. With that said, a **struggle** with **incompatible** smart **plugs**, **internet issues**, as well as **sound sensors not** giving necessary **information** has **delayed** the **project**. These physical constraints directly affect the data quality and reliability of the analysis of the machine learning model.

Economic Constraints:

The total project **budget** is configured to be $**300**, or approximately $**50** per **person** (6 group members), with a target cost for **each unit** being less than **150** dollars. However, as the project continued the **budget** of the project has **increased** and **each** unit's **cost** has **increased** as well. When it comes to the **total cost** of the **budget** we have **not** been **given one**. Even though these constraints have been lightened due to shipping delays we have still had to favor lower-cost, commercially available motion, sound, and **energy-monitoring** devices. While cost is a lighter constraint, the primary objective remains the idea of demonstrating the feasibility of **AI-driven security** and **energy analysis.**

Temporal Constraints:

The project **timelines** is **10**, which requires us to integrate the motion, sound, and energy sensors to allow sufficient data collection to properly **train our machine learning model**. **Hardware** building and **deployment** will be completed by the **end** of **March**, **allowing** for us to **collect** at least a **month** of **meaningful data**. This will **cause** us to have the **machine learning model** be **trained** and **tested**, while data is **still being collected** due to the **condensed timeline**.

1.5 Validation and Testing procedures

This section will outline how we will verify that our smart home system effectively meets the stated goals of optimizing **energy consumption** and enhancing **home security** for

homeowners. The validation process will evaluate whether the system successfully **collects sensor data**, **transfers it securely**, **analyzes** it using **reasoning** techniques, and **provides useful recommendations and alerts** to homeowners. For thorough evaluation, hardware system components will be individually tested, followed by software validation and reasoning functionality tests, and end-to-end system performance under practical**,** realistic conditions.

Hardware Component Testing

Each hardware component of the system will be **individually** tested to verify proper functionality in the context of overall system performance.

- **Sonoff Smart Plugs**: Test accuracy of voltage, current, and power readings under different appliance loads
- **Sound Sensor**: Simulate normal and abnormal sound events to evaluate detection reliability
- **Motion Senso**r: Test home intruder detection under various movement conditions
- **Raspberry Pi 4 Model B**: Verify data collection/polling, local processing, and effective communication with the API server
- **USB-C Power Supply**: Evaluate stability of power delivery to the Raspberry Pi during continuous operation
- **Amazon SD Card**: Test data caching during Wi-Fi network outages and successful synchronization afterward

Software and Reasoning Testing

The software and machine learning portion of the system will be validated by **testing data transfer**, **model reasoning outputs**, and **generation of alerts/recommendations** in the context of **energy optimization** and **security**.

- **Data Processing**: Verify sensor data is received, formatted and transmitted through the API without data corruption
- **Database Reliability**: Confirm data is stored accurately in the database, sensor and device states are logged, and queries return information correctly
- **AI Anomaly Detection**: Introduce scenarios that trigger alerts/recommendations (e.g. energy spikes, unexpected motion, loud noise) and verify that the ML system flags and responds to those events correctly
- **AI Trend Analysis**: Collect data on hourly and multi-day intervals and validate the system identifies patterns in intrusion detection and energy consumption
- **AI Reasoning**: Confirm the AI system generates actionable suggestions for reducing energy consumption and improving security based on trend analysis/anomalies and verify text output
- **Alerts and Notifications**: Verify any alerts or push notifications generated are transmitted through the API and delivered to the user app in real time

- **User Dashboard and App**: Perform unit tests on key user interface features in the app and ensure insights, sensor/device data, and alerts are clearly displayed for interpretation

End-to-End Testing

End-to-end validation confirms that the system operates correctly under realistic home conditions. It verifies the practicality of the system for homeowners and if it is viable for achieving the key objectives of **improving the energy efficiency and security** of a given household.

- **Full Pipeline Verification**: Trigger multiple events at the sensor level and test simultaneous events (i.e. motion and sound sensor detect abnormalities) and confirm data flows correctly through the Raspberry Pi, API, database, AI/ML processing, and to the user application without any corruption
- **Scenario Testing**: Run multiple use cases that reflect the project objectives (energy consumption and security). Each scenario will be tested to verify a user alert or recommendation is received within a certain time interval that is appropriate for the situation. The following are examples of use cases:
    - An appliance plugged into a smart plug is left running for multiple hours and drawing too much power
    - While a homeowner is away from the house, unexpected motion and sound levels are detected at the front door
    - High sounds levels are detected in the house consistent with an appliance breaking or an alarm sound
- **Latency and Reliability Testing**: Measure end-to-end time from event occurrence to user notification and verify continuous monitoring over an extended time period
- **Recovery Testing**: Test Wi-Fi interruption scenarios for brief and long time intervals and confirm the system locally caches data, resumes normal data uploads, and correctly generates recommendations and alerts without significant, permanent data loss

2 Proposed design

2.1 Updates to the proposal design (if applicable)

Since submitting the original project proposal, our design has evolved as a result of weekly team meetings, technical constraints, and a better understanding of what will be required to collect useful data for reasoning. Although our overall concept has remained the same, several implementation details have changed.

Additional findings from the literature and technical documentation

Through further technical documentation review and hands-on evaluation of our hardware options, we discovered that our original sound sensing approach would not provide the type of data needed for meaningful analysis. As a result, we transitioned to **sound microphones** that can plug directly into the Raspberry Pi, allowing us to capture **sound magnitude data** more effectively. This change better supports our goal of **producing usable inputs for contextual reasoning**. Another finding involved the **motion sensors** selected for Units 2 and 3. We discovered that these sensors require a hub in order to operate, which was not initially accounted for in the original proposal. Fortunately, these sensors are **compatible with Home Assistant**, meaning they can still be integrated into the system architecture.

New alternatives considered

As the design matured, we explored several alternatives to improve the **reliability and accessibility of the system**. One major change was moving the database infrastructure to the **AWS cloud**, rather than relying on local storage. Hosting the database in the cloud allows for more **reliable data storage and easier access for analysis**. We also decided to implement **Home Assistant using Docker on the Raspberry Pi**, which provides a more **flexible deployment environment** and allows the devices in the system to be **accessed remotely**. Another alternative involved how we capture electricity consumption data. Initially, we planned to use **smart plugs** to monitor larger appliances. However, we later determined that Unit 3 would instead use **measurement tools connected directly to the electrical breakers**, allowing for more accurate and comprehensive monitoring of household energy usage.

Budget changes and justification

These design updates required adjustments to the project budget. The budget has been extended to support the additional sensing **hardware**, **cloud infrastructure**, and **monitoring tools** needed to capture reliable data. While a final revised total has not been specified, the increase is justified because the original budget did not fully account for the components required to gather **high quality sound, motion, and electricity consumption data** that can support meaningful reasoning within the system.

2.2 System description

The system description will consist of a high-level block diagram, and a functional description of the different parts and interfaces.

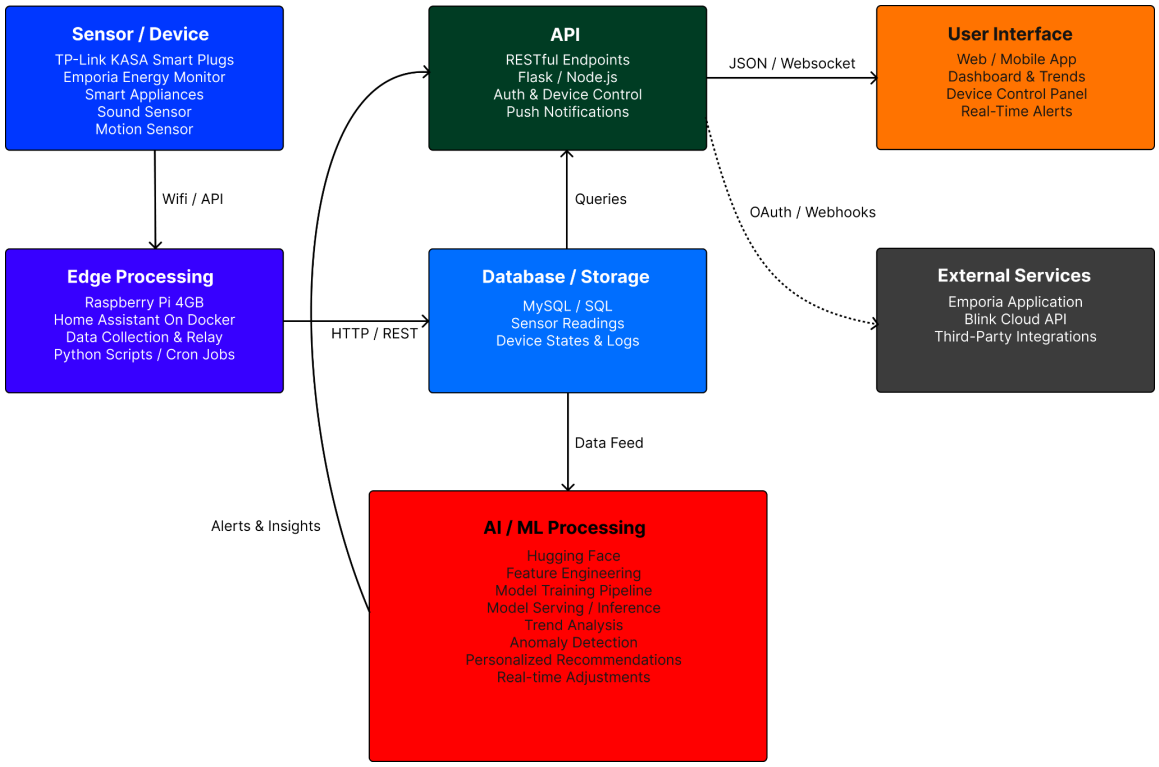

Figure 1: A high level block diagram of the system

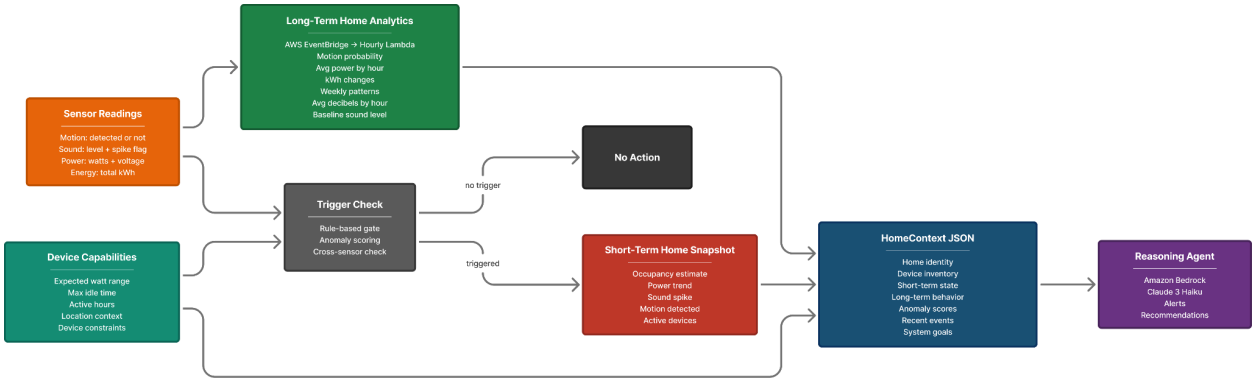

Figure 2: Block Diagram of Machine Learning Architecture

Sensors/Devices

The Sensors and Devices subsystem is responsible for **collecting environmental and energy data** from each of the monitored home units. The data collected will be the basis for the rest of the system. The subsystem includes **energy monitoring devices** (Kasa and Emporia energy monitors), **motion sensors**, and a **sound sending device** (USB microphone), all managed through a **Raspberry PI**.

The energy monitoring devices measure **voltage, current, power, and total energy consumption** at the appliance level with the smart plugs and the breaker level with the Emporia system. Data is collected and periodically sent to our AWS database.

The **motion sensors** detect occupancy using infrared sensing and output a **binary state of motion** (motion detected or no motion detected). These events are timestamped and used for security monitoring and occupancy awareness.

The **sound sensors** are a USB microphone that **detects sound level changes and identifies noise spikes**. These events are logged and used as an additional input for security-related analysis.

The subsystem interfaces with the edge-processing later through **Wifi for the energy devices**, **Zigbee for the motion sensors**, and **USB for the microphone**. The Raspberry PI then communicates with higher-level components via REST APIs and Home Assistant integration.

Edge Processing

The edge processing layer operates on **Raspberry Pi 4B** units (4GB RAM) deployed at each monitored household, running Home Assistant within Docker containers to coordinate and manage all connected smart devices. Each Pi serves as a local processing hub, **collecting real-time data** from Kasa smart plugs, Emporia energy monitors, Zigbee based motion sensors, and USB microphones through Wi-Fi, Zigbee, and USB connections. Python scripts and cron jobs on the Pi handle **local event detection and filtering**, picking up on patterns like unexpected motion or sudden sound spikes, and only passing along meaningful events to the cloud database through the RESTful API. This setup **keeps alert response times under 2 seconds, cuts down on unnecessary bandwidth usage, and allows the system to keep running even if the internet goes out**.

Database/Storage

The database subsystem is responsible for the **storage of system data**. The database stores structured information including **home and device information** and **sensor readings**, that will be used for **data analytics**, **visualization**, and **AI/ML processing**.

The database is a **MySQL relational database** hosted with **Amazon RDS on AWS** providing a **managed database** contained on the cloud. This provides a **scalable** and **reliable** system through **optional upgrades** and **managed backups** able to be connected to from all of the units.

The database is designed to store sensor data at **30 second intervals** per device to ensure efficient inserts and data retrieval.

Data is collected by the sensors, sent through the edge processing subsystem to be stored in the cloud database. That data can then be retrieved by the UI and AI/ML processing subsystems through the API.

The use of **batch inserts** among other methods **improves performance** under write-heavy loads. **Indexing** and efficient querying **improves read performance** for the UI and AI/ML processing subsystems.

API

The API is the **central communication interface** between the subsystems designed to **collect and validate sensor data**, **store data in the database**, and **retrieve data for the UI and AI/ML processing**. It connects the sensors/devices, database, UI, and AI/ML processing.

The API is implemented as a **RESTful API** improving **scalability**, **simplicity**, **flexibility**, and **efficiency**. It is stateless to improve horizontal scalability to improve performance of the system.

The API uses standard HTTP methods such as GET and POST and the data is exchanged using JSON format to create uniformity throughout the API.

AI / ML Processing

The AI/ML pipeline utilizes a multi-stage reasoning architecture hosted on **AWS**, with **Amazon Bedrock** serving as the LLM integration layer and **Claude 3 Haiku** as the reasoning agent. The central goal of this pipeline is to build the most accurate possible context for each home unit before any inference is made, ensuring that the reasoning agent produces reliable, home-specific recommendations and SMS alerts rather than generic responses.

Incoming sensor readings, including motion detection states, sound levels, and smart plug power consumption, first pass through a rule-based **trigger check** that performs anomaly scoring and cross-sensor correlation. This gate determines whether an event warrants further

analysis, preventing unnecessary model calls on routine sensor activity and keeping inference costs low.

When the trigger check is satisfied, the system assembles a **HomeContext JSON payload** by drawing from three supporting tables that together provide a complete picture of the home's current and historical state.

The **Short-Term Home Snapshot** table captures the immediate state of the home, updated continuously as sensor readings arrive. It stores the current occupancy estimate derived from recent motion activity, the active device list, the current power consumption trend over the last 10 minutes, and whether a sound spike has been detected. This table answers the question of what is happening in the home right now.

The **Long-Term Home Analytics** table stores behavioral patterns precomputed hourly by an AWS EventBridge-scheduled Lambda function. Rather than scanning raw sensor readings repeatedly, this table holds aggregated statistics including motion probability by hour of day, average power consumption per hour, weekly device usage patterns, and baseline sound levels. This table answers the question of what is normal for this specific home, which is what allows the reasoning agent to distinguish a genuine anomaly from expected behavior.

The **Device Profiles** table contains static constraints defined per device, including expected watt ranges, maximum idle times, and active hours. These constraints serve as the anomaly thresholds used during the trigger check and provide the reasoning agent with the boundaries within which each device should be operating. This table answers the question of what each device is supposed to be doing.

The reasoning agent receives all three layers of context assembled into a single structured HomeContext JSON document and returns a validated JSON response containing an alert classification, a contextual recommendation, and a reasoning explanation. This output is then routed to the user via SMS notifications and the Home Assistant dashboard.

UI

The user interface subsystem is responsible for **providing occupants with real-time visibility** into the state of their home and **delivering timely notifications when anomalies are detected**. It connects to the rest of the system through the API, receiving sensor data and AI/ML outputs to display to the user.

The primary dashboard is built using **Home Assistant's built-in Lovelace interface**, which runs on the Raspberry Pi and is accessible remotely via **Tailscale**. The Lovelace interface provides customizable cards and views that display live sensor data including energy consumption, motion detection states, and sound levels across all monitored units. This allows occupants to monitor all three units from a single interface without switching between platforms.

For alerting, **SMS notifications** are delivered through a third-party SMS service such as **Twilio** when the AI/ML processing subsystem flags an anomaly. Alerts are triggered by abnormal energy usage, unexpected motion, or sound events that exceed configurable thresholds, giving occupants the ability to define what constitutes an anomaly for their specific household.

Remote access is provided by Tailscale, which creates a secure private network tunnel between the user's device and the Raspberry Pi units. This allows the Home Assistant dashboard to be accessed from outside the home without exposing the system to the public internet.

External Services

The external services subsystem encompasses the third-party platforms and cloud infrastructure that the system depends on for device integration, remote connectivity, and AI/ML processing. It acts as a supporting layer that **extends the capabilities of the edge processing and AI/ML subsystems** beyond what can be achieved locally.

The **TP-Link Kasa integration** is used natively within Home Assistant to communicate with the Kasa smart plugs deployed in Units 1 and 2. Home Assistant polls these devices over the local WiFi network using the Kasa LAN protocol, collecting voltage, current, power, and total energy consumption readings every 30 seconds. The Emporia Energy Monitor connects to Home Assistant through the Emporia integration and provides breaker-level energy consumption data for Unit 3, giving the system per-circuit visibility into whole-home power draw.

Tailscale provides the secure networking layer that links all three Raspberry Pi units together and enables remote access to the Home Assistant dashboard. Rather than relying on port forwarding or a public IP address, Tailscale creates an encrypted peer-to-peer mesh network that keeps all inter-unit communication private.

Amazon Web Services hosts two core components of the system. The MySQL database is managed through Amazon RDS as described in the database subsystem. The AI/ML pipeline is hosted on AWS using Amazon Bedrock as the LLM integration layer, with AWS EventBridge and Lambda handling scheduled hourly analytics jobs that precompute motion baselines, average power profiles, and sound level benchmarks. These precomputed analytics are consumed by the reasoning agent at inference time to provide historical context alongside real-time sensor data. All device data collected by Home Assistant is forwarded to the AWS RDS database through the RESTful API, making AWS the convergence point for both storage and reasoning across all three units.

2.3 Complete module-wise specifications

Sensors/Devices

The sensors and devices subsystem consists of energy monitoring devices, motion sensors, and a sound sensing device, all interfaced through a Raspberry Pi edge node. Energy data is collected using TP-Link Kasa smart plugs and an Emporia Vue Energy Monitor, while motion detection is performed using Zigbee-based motion sensors. Sound sensing is implemented using a USB microphone connected directly to the Raspberry Pi.

All devices are commercial off-the-shelf components and do not require custom circuit design. Instead, the hardware design focuses on system integration and communication with the Raspberry Pi. The Raspberry Pi GPIO structure, shown in Figure 2, provides expansion capability for future sensor integration.

The Kasa smart plugs are deployed in Units 1 and 2 and report voltage, current, power, and total energy consumption at the appliance level. The Emporia Vue Energy Monitor is deployed in Unit 3 and is installed directly at the breaker panel, with current transformer clamps attached to individual breakers to capture per-circuit power draw across the entire home. This provides a more comprehensive view of whole-home energy consumption than plug-level monitoring alone.

Motion detection is performed using Haozee Zigbee-based presence detectors that use passive infrared sensing to detect body heat. Each sensor outputs a binary motion state indicating whether motion is detected or not, which is timestamped and logged to the database for security monitoring and occupancy inference. The sensors communicate with the Raspberry Pi through a Sonoff Zigbee 3.0 USB dongle, which acts as the Zigbee coordinator and bridges the Zigbee network to Home Assistant over USB.

Sound sensing is implemented using a USB microphone that monitors ambient noise levels continuously. A Python script running on the Raspberry Pi samples the microphone input and logs an event when the sound level exceeds a configurable threshold, capturing the magnitude of the spike. This allows the system to detect security-relevant audio events such as glass breaking or sudden loud noises without storing continuous audio recordings.

Devices communicate with the Raspberry Pi using the interfaces summarized in Table 3. Energy devices connect over WiFi, motion sensors communicate through the Zigbee USB dongle, and the microphone connects via USB. Energy data is sampled every 30 seconds, while motion and sound sensors operate in an event-driven manner, generating records only when activity is detected. All sensor data is forwarded from the Raspberry Pi to the cloud database through the RESTful API following the flow below.

Sensors → Raspberry Pi → Home Assistant / API → Database

Edge Processing

In our system, edge processing plays a critical role by enabling local decision-making for real-time security events. Instead of relying on cloud communication, alerts and automated actions are handled directly on the device, significantly reducing latency and ensuring faster response times. This also allows the system to continue functioning even if the internet connection is lost, which is essential for maintaining reliable home security. At the edge, we run a Dockerized instance of Home Assistant that serves as the central coordination and automation engine. It collects and filters sensor data from motion detectors, sound sensors, and cameras, processing these inputs locally to determine whether an event is significant.

Event detection logic, implemented through Python scripts and scheduled tasks (such as cron jobs), analyzes patterns like unexpected motion or sudden noise spikes to identify potential threats. Rather than transmitting all raw data, the system only relays relevant events to the cloud or dashboard, which improves efficiency and reduces unnecessary data transfer. This approach not only speeds up alerts but also enhances privacy, as sensitive information remains local, and lowers overall bandwidth usage.

To validate the system, we conducted end-to-end testing using simulated real-world home scenarios, including motion, sound, and energy-related events. These tests verified the complete pipeline—from sensors to Home Assistant, through the API, and into the database. Key tests included motion detection with controlled movement at varying angles and distances, sound detection by comparing normal background noise to abnormal spikes, and energy monitoring to ensure accurate tracking and response.

Database/Storage

The database system is implemented as a MySQL relational database hosted with Amazon RDS on AWS. It uses the t4g micro instance with 20 GiB of gp3 SSD storage that can be increased if needed.

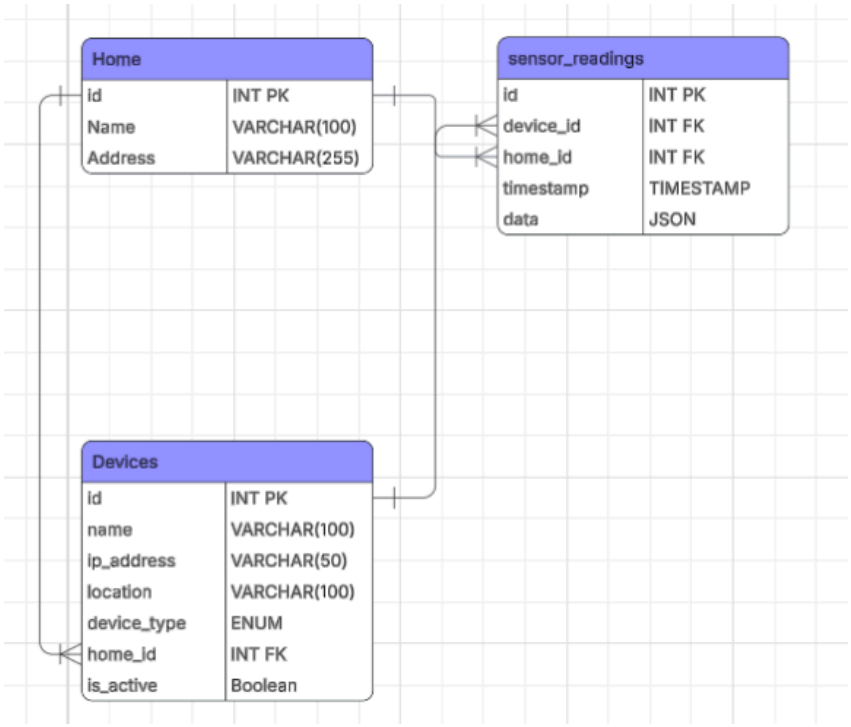

Figure 3: ER diagram for database

For sensor data storage, the database contains 3 tables:

- Homes
- Devices
- Sensor Readings

The homes table contains some basic information used to identify the home, such as the name and address of the home.

The devices table contains more detailed information about each device to help identify it and better organize data for analytics. It contains the name and ip_address which are used to identify the device. The location and device_type are used to organize devices. The home_id is a reference to the home the device is contained within. Finally, is_active will allow the system to more easily toggle devices on and off and keep track of it.

The sensor readings table is where the core of the data is stored. It contains the home_id and device_id to reference where the data is coming from. The data is stored in JSON format to allow for flexible data sets as different devices may send different kinds of data. Lastly, the timestamp of when the data was collected.

API

The API subsystem is implemented as a RESTful API and serves as the central communication interface between all other subsystems. It is responsible for receiving and validating sensor data from the edge processing layer, storing that data in the database, and providing data access to

the UI and AI/ML processing subsystems. All data exchange across the API is performed using JSON to ensure uniformity and compatibility across subsystems.

The API is stateless, meaning each request contains all the information needed to process it without relying on stored session state. This design improves horizontal scalability and simplifies the overall system since no session management infrastructure is required. Standard HTTP methods are used throughout: POST requests are used for data ingestion and device registration, while GET requests are used for data retrieval by the UI and AI/ML processing subsystems.

The API exposes four core endpoints. The /devices/list endpoint accepts GET requests and returns a list of all registered devices in the system. The /devices/add endpoint accepts POST requests and registers a new device by recording its name, IP address, location, device type, home association, and active status in the devices table. The /readings/add endpoint accepts POST requests and performs batch inserts of sensor readings into the sensor readings table, supporting multiple readings per request to reduce write overhead under high-frequency polling loads. The /readings/list/<device_id> endpoint accepts GET requests and returns all historical readings associated with a given device, which is used by the UI for trend visualization and by the AI/ML processing subsystem for context assembly.

All requests are validated before any database interaction occurs to prevent malformed or incomplete data from being written to the database. Responses follow standard HTTP status codes, with errors returned as structured JSON objects to allow calling subsystems to handle failures gracefully.

AI/ML Processing

The AI/ML processing subsystem is responsible for analyzing sensor data collected from all three units and producing actionable security alerts and energy recommendations for the occupant. The pipeline is built around a multi-stage reasoning architecture that is designed to minimize unnecessary inference calls while ensuring meaningful events are always analyzed.

Incoming sensor readings from the database first pass through a rule-based trigger check that performs anomaly scoring and cross-sensor correlation. This gate evaluates each new reading against device profiles containing static constraints such as expected watt ranges, maximum idle durations, and expected active hours. If a reading falls outside normal bounds, or if multiple sensors fire within a short time window such as simultaneous motion and sound detection, the trigger check flags the event for further analysis. Readings that do not meet the trigger threshold result in no action, keeping inference costs low and system performance high.

When an event is triggered, the system assembles a HomeContext JSON payload that provides the reasoning agent with the full context needed to evaluate the event. The payload combines a short-term home snapshot capturing current occupancy estimates, active devices, recent power trends, and sound spike flags, with long-term analytics precomputed on an hourly schedule.

These long-term analytics include motion probability baselines by hour, average power consumption per circuit, weekly usage patterns, and baseline sound levels. Together they allow the reasoning agent to distinguish between behavior that is unusual relative to the current moment and behavior that is unusual relative to established historical norms.

The reasoning agent is a large language model that receives the HomeContext payload and returns a structured JSON response containing three fields: an alert classification indicating the severity and type of event, a contextual recommendation describing the suggested action for the occupant, and a reasoning explanation describing why the event was flagged. This structured output is then routed to the user via SMS through Twilio and surfaced on the Home Assistant Lovelace dashboard.

The AI/ML subsystem is validated by simulating known anomaly scenarios and verifying that the agent produces correct and consistent structured outputs. For energy monitoring, this includes simulating a device exceeding its maximum idle power threshold and a normal HomeContext with no anomalies present to confirm the system does not produce false positives. For security monitoring, this includes simulating unexpected motion during overnight hours with occupancy set to likely empty, and simulating a concurrent sound spike and motion event. Validation is considered successful if the agent accurately classifies each scenario, correctly distinguishes between an event-driven alert and a general recommendation, and always returns a valid JSON response containing all three required fields.

UI

The user interface subsystem is responsible for providing occupants with real-time visibility into the state of their home and delivering timely notifications when anomalies are detected.

The primary dashboard is built using Home Assistant's built-in Lovelace interface, which runs on the Raspberry Pi and is accessible remotely via Tailscale. The Lovelace interface provides customizable cards and views that display live sensor data including energy consumption, motion detection states, and sound levels across all monitored units. This allows occupants to monitor all three units from a single interface without switching between platforms.

For alerting, SMS notifications are delivered through a third-party SMS service such as Twilio when the AI/ML processing subsystem flags an anomaly. Alerts are triggered by abnormal energy usage, unexpected motion, or sound events that exceed configurable thresholds. This gives occupants the ability to define what constitutes an anomaly for their specific household. Alerts and dashboard updates are delivered to the user via the API using JSON over WebSocket.

Remote access is provided by Tailscale, which creates a secure private network tunnel between the user's device and the Raspberry Pi units. This allows the Home Assistant dashboard to be accessed from outside the home without exposing the system to the public internet.

External Services

The external services subsystem encompasses the third-party platforms and cloud infrastructure that the system depends on for device integration, remote connectivity, and AI/ML processing.

The TP-Link Kasa integration is used natively within Home Assistant to communicate with the Kasa smart plugs deployed in Units 1 and 2. Home Assistant polls these devices over the local WiFi network using the Kasa LAN protocol, collecting voltage, current, power, and total energy consumption readings every 30 seconds. The Emporia Energy Monitor connects to Home Assistant through the Emporia integration and provides breaker-level energy consumption data for Unit 3, giving the system per-circuit visibility into whole-home power draw.

Tailscale provides the secure networking layer that links all three Raspberry Pi units together and enables remote access to the Home Assistant dashboard. Rather than relying on port forwarding or a public IP address, Tailscale creates an encrypted peer-to-peer mesh network that keeps all inter-unit communication private.

Amazon Web Services hosts two core components of the system. The MySQL database is managed through Amazon RDS as described in the database subsystem. The AI/ML pipeline is hosted on AWS using Amazon Bedrock as the LLM integration layer, with AWS EventBridge and Lambda handling scheduled hourly analytics jobs that precompute motion baselines, average power profiles, and sound level benchmarks. These precomputed analytics are consumed by the reasoning agent at inference time to provide historical context alongside real-time sensor data.

All device data collected by Home Assistant is forwarded to the AWS RDS database through the RESTful API, making AWS the convergence point for both storage and reasoning across all three units.

Table 1: A parts list of materials that are needed to build the design. Note that the label and caption are above the table. The specific format of the table is flexible, I have given a basic example to illustrate labeling and captioning.

	Amount	Amount per Unit	Cost per Unit	Total Amount ($)
Kasa Smart Plug Mini	10	5	14.99	150
Amazon SD Card	2	1	6.25	13.50
Raspberry Pi (One Free)	3	1	75	150
Emporia Smart Plugs (Unit 3)	16	16	8.75	140
Raspberry PI 15 W USB-C Power Supply	3	1	8	16
Suzie Pi 2Pcs	2	1	4.87	9.73

Mini Microphone				
Haozee Smart Motion Sensor	2	1	18	36
Sonoff Zigee 3.0 USB Dongle	2	1	40	80
Emporia Energy Monitor	1	1	200	200
AWS	1	1	X	11
Total Cost	42	13-29	375.86	849.23

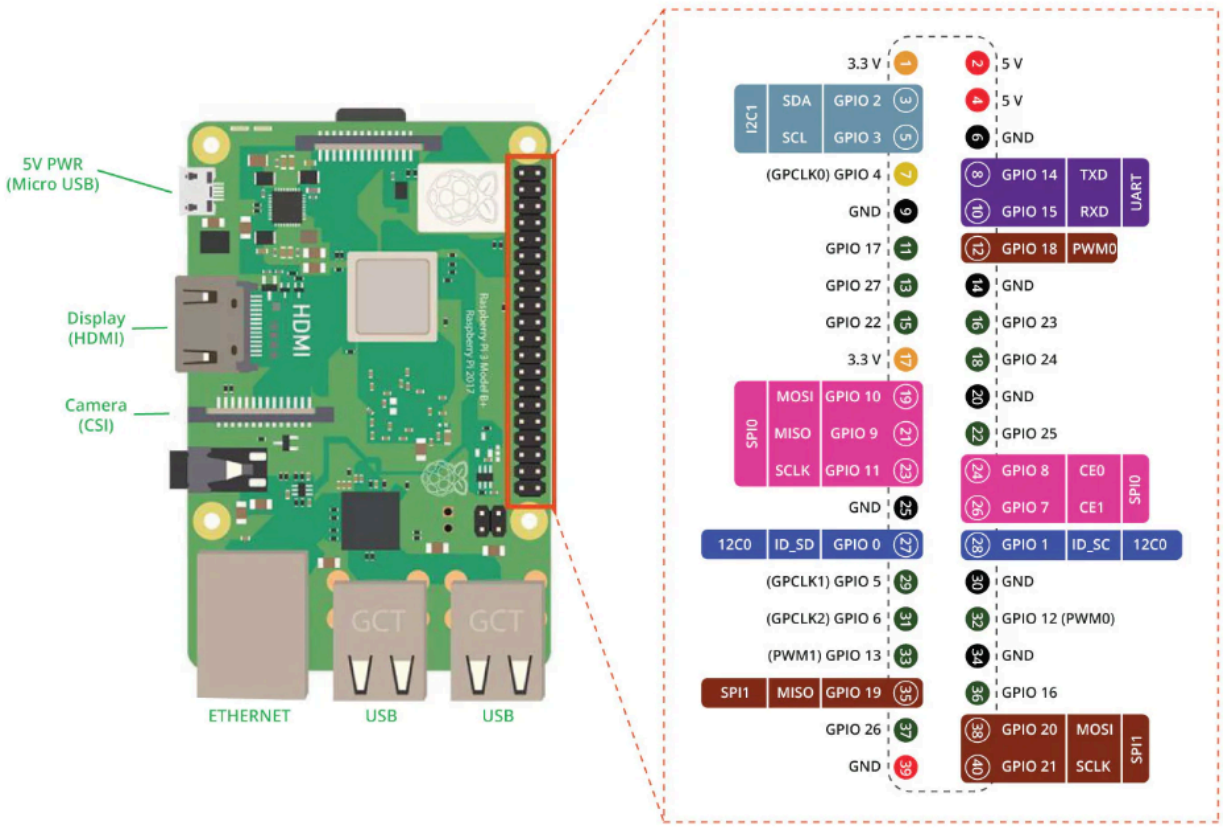

Figure 4: Raspberry Pi 4 Model B GPIO (J8) Pinout Diagram showing the 40 pin interface including power pins, ground pins, general purpose input and output pins, and communication interfaces such as I2C, SPI, and UART used for connecting external sensors and peripherals.

Table 2: Raspberry Pi 4 Model B Digital I/O Pin AC Characteristics

Pin Name	Symbol	Parameter	Minimum	Typical	Maximum	Unit
Digital outputs	t_{rise}	10-90% rise time[a]	-	TBD	-	ns
Digital outputs	t_{fall}	90-10% fall time[a]	-	TBD	-	ns

[a] Default drive strength, CL = 5pF, VDD_IO = 3.3V

Figure 5: Raspberry Pi 4 Model B Digital I/O Diagram

Updated Hardware Overview

The Sensors and Devices subsystem consists of energy monitoring devices, motion sensors, and a sound sensing device, all interfaced through a Raspberry Pi edge node. Energy data is collected using TP-Link Kasa smart plugs and an Emporia Vue Energy Monitor, while motion detection is performed using Zigbee-based motion sensors. Sound sensing is implemented using a USB microphone connected directly to the Raspberry Pi.

All devices are commercial off-the-shelf components and do not require custom circuit design. Instead, the hardware design focuses on system integration and communication with the Raspberry Pi. The Raspberry Pi GPIO structure, shown in Figure 2, provides expansion capability for future sensor integration.

Interfaces and Communication

Devices communicate with the Raspberry Pi using WiFi, Zigbee, and USB interfaces. Energy devices connect over WiFi, motion sensors communicate through a Zigbee USB dongle, and the microphone connects via USB.

Table 3: Interfaces for Sensors and Devices Subsystem

Device	Interface	Output
Kasa / Emporia	Wifi	Voltage, Current, Power, Total Energy Consumption (kWh)
Motion Sensor	Zigbee	Motion (binary)

Microphone	USB	Sound level / Spike

Timing Behavior

The subsystem uses both periodic and event-driven operation. Energy data is sampled every 30 seconds, while motion and sound sensors generate events only when activity is detected. This allows efficient data collection while maintaining real-time responsiveness for security monitoring.

Data Flow

All sensor data is first collected by the Raspberry Pi and then forwarded to external systems for storage and processing.

Sensors → Raspberry Pi → Home Assistant / API → Database

3 Project management

3.1 Updated implementation schedule

Table 4: Implementation Schedule and task dependencies with task status, start dates, and due dates.

	Task Name	Dependencies	Status	Start Date	Due Date
1	**Unit 1 and 2 Setup**	None	Complete	02/26/26	03/06/26
2	**Unit 3 Setup**	None	In Progress	03/05/26	03/20/26
3	**Home Assistant Configuration**	Task 1, Task 2	Complete	03/01/26	03/05/26
4	**Database Setup**	Task 1, Task 2	Complete	02/28/26	03/05/26
5	**Data Collection**	Task 1, Task 2, Task 3	In Progress	03/06/26	04/18/26
6	**API Setup**	Task 1, Task 2, Task 3	In Progress	03/08/26	03/20/26
7	**Database Refinement**	Task 3, Task 4	Not Started	03/18/26	03/28/26
8	**AI/ML Module**	Task 5, Task 7	In Progress	03/18/26	04/05/26

9	**User Interface**	Task 4, Task 5	In Progress	03/18/26	04/01/26
10	**Testing/Debugging**	Task 1-9	Not Started	04/05/26	04/20/26

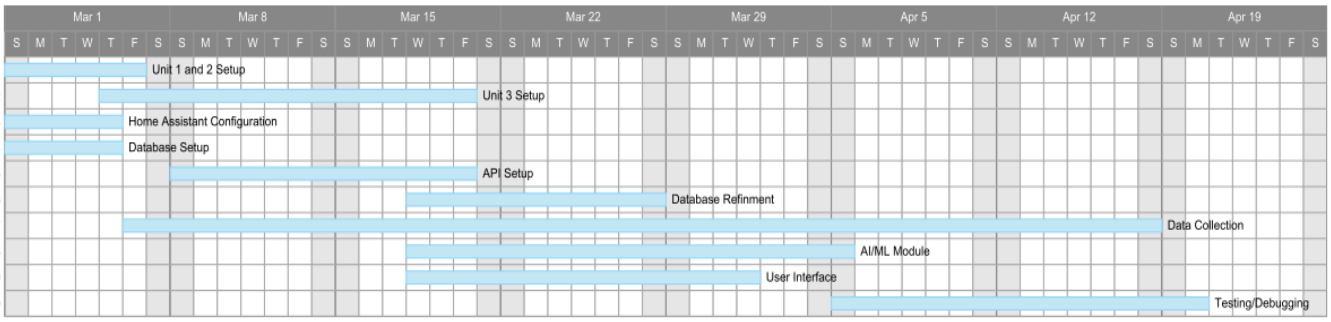

Figure 6: *Gantt Chart representing feasible Project timeline. The critical path of the project runs through Unit Setup, Home Assistant Configuration, API Setup, AI/ML Module development, and Testing/Debugging. Delays in any of these tasks would directly impact the final project completion date, making them the highest priority tasks during implementation.*

After **Database Setup** is completed, several development tasks begin in parallel. **Data Collection** starts first and depends on Task 1, Task 2, and Task 3, allowing sensor data from the configured devices to begin flowing into the system. At the same time, **API Setup** begins, which also depends on the initial setup and Home Assistant configuration. This API layer allows the system to retrieve and interact with the collected sensor data stored in the database. Following the database implementation, **Database Refinement** begins and depends on both Home Assistant Configuration and the initial Database Setup. This step focuses on improving database structure, optimizing queries, and ensuring the system can efficiently store and retrieve the increasing amount of sensor data.

The **AI/ML Module** depends on both **Data Collection** and **Database Refinement**, since it requires a stable data pipeline and structured database schema before reasoning models can be implemented. This module processes the collected data to generate contextual insights for the smart home system. The **User Interface** begins after the database and initial data collection are available. It depends on Database Setup and Data Collection so that the interface can display real sensor data and system insights to the user. The **critical path** after Database Setup follows:

Unit Setup → Database Setup → Database Refinement → AI/ML Module → Testing and Debugging

This path determines the overall completion time because each task depends on the previous one and they collectively extend to the latest completion date. If **Database Refinement** or the **AI/ML Module** is delayed, the start of **Testing and Debugging** will also be delayed, directly impacting the final project timeline. Although **API Setup**, **User Interface**, and **Data Collection** are essential for system functionality, they do not extend the total project duration beyond the critical path. The final stage, **Testing and Debugging**, integrates all components including the

database, AI/ML reasoning module, API services, and user interface to ensure the entire system functions correctly before project completion.

Phase 1 Milestone (Feb 26)

Phase 2 – Data and Intelligence Layer (Feb 26 – Mar 24)

1. **Data Collection (Mar 6 – April 18)**
    - Logging sensor streams
    - Cleaning and formatting data
    - Initial dataset validation
2. **API Testing (Mar 8 – March 20)**
    - Verify API endpoints data
    - Test JSON request and response formatting
    - Validate database storage
    - Check error handling and response behavior
3. **AI/ML Module (Mar 18 – April 5)**
    - Feature extraction
    - Model selection
    - Training and tuning
    - Local integration testing
4. **User Interface (Mar 18 - April 1)**
    - Dashboard layout
    - Alert logic integration
    - Backend connection

CDR Report (Mar 18)

Phase 2 Milestone (Mar 24)

Phase 3 – Integration and Delivery (April 5 - April 27)

1. **Testing and Debugging (April 5 – Apr 20)**
    - System integration testing
    - Performance validation
    - Edge case handling

Phase 3 Milestone (Apr 20)

Project Demo (Apr 20)

Presentation preparation

Demo environment setup

Final Report (Apr 27)

3.2 Updated validation and testing procedures

Smart Plug Energy Monitoring

To ensure smart plugs are operating as expected and with accuracy, energy data is collected from the smart plugs and compared against expected appliance energy data.

The system is considered valid if the collected data falls within +/- 5% of the expected value. The smart plugs must also collect accurate data at a consistent basis.

Motion Sensor Detection

Motion detection is validated by simulating movement from different angles, positions, and at different speeds.

The system meets validation requirements if motion is accurately detected more than 90% of the time.

Sound Sensor Detection

Sound detection is evaluated by simulating both normal and abnormal sounds, including ambient noise and sudden high-intensity sounds such as doors slamming, glass breaking, etc.. The goal is to verify that the system can distinguish between typical background noise and potentially significant events.

Validation is achieved if the sensor successfully differentiates between normal and abnormal sound levels and correctly triggers events when thresholds are exceeded.

End-to-End System Testing

System-level validation is performed by simulating realistic home scenarios involving combinations of motion, sound, and energy events. These tests verify the complete data pipeline, including:

Sensors → Home Assistant → API → Database

The system is considered successful if all components operate together without failure and data flows correctly through each stage of the pipeline.

Data Ingestion and Storage

Data ingestion is validated by verifying that sensor readings are received every 30 seconds by comparing the timestamps of incoming data.

The system meets validation criteria if at least 98% of expected readings are successfully received.

Data storage is validated by inserting known test data into the system and querying the database. The system must demonstrate 100% data accuracy with no loss of data.

System Performance Testing

Performance testing is conducted by simulating multiple devices transmitting data simultaneously to evaluate system scalability and responsiveness.

The system meets performance requirements if:

- Event detection latency is less than 2 seconds
- Data insertion latency is less than 1 second per reading
- The system maintains stable operation under continuous data collection

AI/ML Validation

The AI/ML components are validated using simulated scenarios for both energy and security applications.

For energy monitoring, abnormal power usage is simulated, including devices exceeding predefined idle thresholds, and compared against normal operating conditions.

For security monitoring, scenarios such as unexpected motion during unoccupied periods and concurrent motion and sound events are simulated.

Validation is achieved if the system:

- Accurately identifies anomalies
- Generates appropriate recommendations based on event type and urgency
- Correctly distinguishes between alerts and general recommendations
- Produces structured outputs containing alert, recommendation, and reasoning fields

User Interface and Notification Testing

The user interface is validated by verifying that real-time sensor data is correctly displayed on the dashboard. Additionally, known anomalies are triggered to test the notification system.

The system meets validation criteria if:

- Data updates occur without stale or missing readings
- Notifications (e.g., SMS alerts) are received within a reasonable time following event detection

Overall System Validation

The system is considered fully validated when it demonstrates:

- Accurate detection across all sensor types (≥ 90%)
- Energy measurement accuracy within ±10%
- Reliable data transmission with minimal loss
- Correct alert generation for abnormal events
- Near real-time system responsiveness

3.3 Updated division of labor and responsibilities

William | Frontend Software Engineer

- Established and configured the GitHub integration with Tailscale to streamline version control and secure network access for the development team.
- Deployed and configured the Home Assistant platform within a Docker container on the Raspberry Pi designated for Unit 1, enabling centralized smart home management.
- Initiated frontend development efforts, laying the groundwork for the user-facing interface of the application.
- Provisioned and connected Emporia smart plugs to the Unit 3 environment and integrated the corresponding Home Assistant extension for Emporia device management.
- TBD Depends on the mac mini

Sriharsha | *Machine Learning Engineer*

- Architected and provisioned the AWS cloud infrastructure to support model training, data storage, and deployment pipelines.
- Authored comprehensive documentation detailing the AI models selected for implementation, including their intended roles within the system's reasoning framework.
- Refined and optimized data collection intervals to ensure that data retention policies are aligned with the analytical and reasoning requirements of the project.
- Integrated and mapped Emporia smart plugs and the associated breaker panel to their corresponding Home Assistant dashboard cards for real-time energy monitoring.
- TBD Depends on the mac mini

Shiv | *Lead Systems Engineer*

- Provided ongoing project oversight to ensure all team milestones were met on schedule, with regular updates to the project timeline reflecting current progress.
- Maintained consistent and transparent communication across all team members

to ensure alignment on objectives, deliverables, and task dependencies.
- Deployed and configured the Home Assistant platform within a Docker container on the Raspberry Pi designated for Unit 2.
- Provisioned and connected the Emporia smart plugs for Unit 3 to the Emporia cloud platform for remote energy monitoring and management.
- TBD Depends on the mac mini

Tyler | ***Hardware and Embedded Systems Engineer***

- Completed the physical installation and configuration of all sensors and power monitoring devices within Unit 1.
- Established connectivity between the Kasa smart plugs and the companion mobile application, enabling remote device control and status monitoring.
- Deployed and configured the Home Assistant platform within a Docker container on the Raspberry Pi designated for Unit 3.
- Verified and validated that sensor data from Unit 1 was being collected accurately and transmitted consistently to the data pipeline.
- Integrated and mapped Emporia smart plugs and the associated breaker panel to their corresponding Home Assistant dashboard cards for Unit-level energy visibility.
- TBD Depends on the mac mini

Sutton | ***Hardware / Machine Learning Engineer***

- Completed the physical installation and configuration of all sensors and power monitoring devices within Unit 2.
- Established connectivity between the Kasa smart plugs and the companion mobile application for Unit 2, enabling remote control and monitoring capabilities.
- Verified and validated that sensor data from Unit 2 was being uploaded accurately and consistently to the centralized database.
- Developed and deployed custom automation scripts for the motion sensor and Suzie Microphone hardware components across Unit 2 and Unit 3.
- TBD Depends on the mac mini

Evan | ***Database and Backend Engineer***

- Configured and deployed the SQL database on AWS, ensuring secure cloud-based storage with appropriate access controls and encryption protocols.
- Developed and implemented RESTful API endpoints to facilitate the transmission of sensor data from the Raspberry Pi units in Unit 1 and Unit 2 to

the centralized database.

- Customized and extended the database schema to accommodate the Unit 3 Emporia breaker box and smart plug configuration, ensuring seamless data ingestion and consistency across all monitored units.
- TBD Depends on the mac mini

4 Preliminary results

Within the system, the majority of hardware components have been set up, with the exception of Unit 3, which requires additional development. Home Assistant has been configured to support the collection and organization of data from the sensors. Python scripts have been developed to perform edge processing on data collected from the active hardware components in Units 1 and 2. The MySQL database has been deployed and hosted on AWS to enable reliable storage of sensor data. API endpoints have been implemented to support the ingestion and storage of data collected by the sensors, with additional endpoints to be added as needed. The UI and AI/ML processing components are pending further hardware finalization and sufficient data collection before implementation begins.

Data has begun to be collected in units 1, and home and device information for units 1 and 3 have been collected.

id	name	address
1	Sutton's Home	Medium Home
2	Will's Home	Medium Home
3	Professor's Home	Large Home

Figure 7: Homes table within database

id	name	ip_address	location	device_type	home_id	is_active
1	AirFryer Plug	192.168.1.104	Kitchen	smart_plug	1	1
2	Microwave Plug	192.168.1.102	Kitchen	smart_plug	1	1
3	Washer Plug	192.168.1.226	Kitchen	smart_plug	1	1
4	Bedroom Powerstrip Plug	192.168.1.16	Bedroom	smart_plug	1	1
5	Living Room Setup Plug	192.168.1.55	Living Room	smart_plug	1	1
6	USB Microphone	local_pi	Living Room	sound_sensor	3	1
7	Breaker 1	emporia	Panel	smart_plug	3	1
8	Breaker 2	emporia	Panel	smart_plug	3	1
9	Breaker 3	emporia	Panel	smart_plug	3	1
10	Breaker 4	emporia	Panel	smart_plug	3	1
11	Breaker 5	emporia	Panel	smart_plug	3	1
12	Breaker 6	emporia	Panel	smart_plug	3	1
13	Breaker 7	emporia	Panel	smart_plug	3	1
14	Breaker 8	emporia	Panel	smart_plug	3	1
15	Breaker 9	emporia	Panel	smart_plug	3	1
16	Breaker 10	emporia	Panel	smart_plug	3	1
17	Breaker 11	emporia	Panel	smart_plug	3	1
18	Breaker 12	emporia	Panel	smart_plug	3	1
19	Breaker 13	emporia	Panel	smart_plug	3	1
20	Breaker 14	emporia	Panel	smart_plug	3	1
21	Breaker 15	emporia	Panel	smart_plug	3	1
22	Breaker 16	emporia	Panel	smart_plug	3	1
23	Emporia Plug1	emporia	Unknown	smart_plug	3	1
24	Emporia Plug 2	emporia	Unknown	smart_plug	3	1
25	Emporia Plug 3	emporia	Unknown	smart_plug	3	1
26	Emporia Front Door	emporia	Front Door	smart_plug	3	1
27	Motion Sensor	local_ha	Living Room	motion_sensor	3	1

Figure 8: Devices table within database

id	device_id	home_id	timestamp	data
222467	25	3	2026-03-19 04:04:18	{"power": 0.0, "current": 0.0, "voltage": 120.0}
222468	26	3	2026-03-19 04:04:18	{"power": 0.0, "current": 0.0, "voltage": 120.0}
222466	24	3	2026-03-19 04:04:18	{"power": 0.0, "current": 0.0, "voltage": 120.0}
222459	17	3	2026-03-19 04:04:17	{"power": 0.0, "current": 0.0, "voltage": 120.0}
222456	14	3	2026-03-19 04:04:17	{"power": 0.0, "current": 0.0, "voltage": 120.0}
222461	19	3	2026-03-19 04:04:17	{"power": 0.0, "current": 0.0, "voltage": 120.0}
222458	16	3	2026-03-19 04:04:17	{"power": 0.0, "current": 0.0, "voltage": 120.0}
222464	22	3	2026-03-19 04:04:17	{"power": 0.0, "current": 0.0, "voltage": 120.0}
222465	23	3	2026-03-19 04:04:17	{"power": 49.245, "current": 0.4104, "voltage": 120.0}
222462	20	3	2026-03-19 04:04:17	{"power": 0.0, "current": 0.0, "voltage": 120.0}
222460	18	3	2026-03-19 04:04:17	{"power": 109.0722, "current": 0.9089, "voltage": 120.0}
222463	21	3	2026-03-19 04:04:17	{"power": 0.0, "current": 0.0, "voltage": 120.0}
222457	15	3	2026-03-19 04:04:17	{"power": 0.0, "current": 0.0, "voltage": 120.0}
222451	9	3	2026-03-19 04:04:16	{"power": 77.3015, "current": 0.6442, "voltage": 120.0}
222454	12	3	2026-03-19 04:04:16	{"power": 0.0, "current": 0.0, "voltage": 120.0}
222455	13	3	2026-03-19 04:04:16	{"power": 0.0, "current": 0.0, "voltage": 120.0}
222453	11	3	2026-03-19 04:04:16	{"power": 61.9489, "current": 0.5162, "voltage": 120.0}
222449	7	3	2026-03-19 04:04:16	{"power": 912.4283, "current": 7.6036, "voltage": 120.0}
222452	10	3	2026-03-19 04:04:16	{"power": 0.0, "current": 0.0, "voltage": 120.0}
222450	8	3	2026-03-19 04:04:16	{"power": 0.0, "current": 0.0, "voltage": 120.0}

Figure 9: Sensor_readings table within database

5 References

https://ieeeauthorcenter.ieee.org/wp-content/uploads/IEEE-Reference-Guide.pdf

Raspberry Pi Foundation, *Raspberry Pi 4 Model B Datasheet*, RP-008341-DS, Version 1. Raspberry Pi Foundation. Available:

https://pip-assets.raspberrypi.com/categories/545-raspberry-pi-4-model-b/documents/RP-008341-DS-1-raspberry-pi-4-datasheet.pdf. Accessed: Mar. 10, 2026.